

Project Architecture Report

Student name & ID (שמות הסטודנטים · ת.ז.)	Yuval Bershtansky · 328274949 (solo project)
Project code / group (קוד פרויקט)	19
Mentor (מנחה)	Dr. Ariel Rot
Project title	Smart Traffic Intersection Management System

Abstract

This project implements a **Smart Traffic Intersection Management System**: a Deep Reinforcement Learning controller that dynamically selects the green-light phase at a single four-way intersection in order to reduce vehicle waiting time and improve throughput. A **plain Deep Q-Network (DQN)** agent chooses, every 10 seconds, one of eight NEMA dual-ring signal phases, based on the live traffic state of the junction. The reinforcement-learning state is augmented with a **frozen LSTM forecast** of the near-future queue lengths (60/90/120 seconds ahead), forming a hybrid predictive-plus-reactive controller. The system is built on the **SUMO** microscopic traffic simulator (driven from Python via the TraCI interface) and is evaluated against three established non-RL baselines — Webster fixed-time, max-pressure, and SUMO's native actuated control — under a multi-seed, statistically rigorous protocol (paired Wilcoxon signed-rank tests with Holm–Bonferroni correction). The architecture is organised as five layers around a central Python backend hub, with a SQLite/JSONL persistence layer, a full provenance chain for reproducibility, and a real-time visualisation layer (React dashboard + Unity 3D). The project is a **replication-plus-adaptation** of MPLight (Chen et al., 2020) rather than novel research, and is deliberately simulation-only and single-user.

1. Introduction

Urban traffic congestion is dominated by how efficiently signalised intersections are controlled. Classical fixed-time controllers (e.g. Webster) cannot adapt to fluctuating demand, while purely reactive adaptive controllers act only on the present and ignore where traffic is heading. Reinforcement learning offers a way to *learn* a control policy directly from simulated experience, and recent work (MPLight, PressLight) shows that a compact "pressure"-based state is both effective and transferable. This project designs and builds such a controller for one four-way intersection, and goes one step further by feeding the agent a short-term **learned forecast** of the queues, so the policy can act on what is *about* to happen. The work emphasises a clean, layered software architecture, deterministic reproducibility, and an honest empirical comparison against strong baselines — the qualities expected of an engineered final-year project rather than an ad-hoc script.

2. Problem Statement

A signal controller at a four-way intersection must repeatedly decide which movements receive a green light, under hard safety constraints (minimum/maximum green time, mandatory yellow and all-red clearance, and NEMA dual-ring phase legality). Fixed-time plans are computed offline and never adapt; greedy adaptive rules (e.g. max-pressure) optimise only the current instant and can be short-sighted; and naive RL controllers can silently violate safety timing or learn biased value functions if the legal-action constraints are not enforced correctly. The challenge this project addresses is: **can a DQN agent, augmented with a learned short-term traffic forecast, control a single intersection at least as well as — and ideally better than — three established baselines, while respecting all safety constraints and remaining fully reproducible?** The answer must be demonstrated with statistically sound evidence, not a single lucky run.

3. Objectives

1. Build a deterministic SUMO simulation of a four-way, 12-movement intersection, controlled from Python via TraCI with a Gym-style environment API.
2. Implement safety-correct signal control: action masking (min-green 10 s / max-green 60 s / NEMA barrier legality) plus yellow (3 s) and all-red (2 s) transitions enforced in the environment.
3. Train a plain DQN agent that selects one of eight NEMA phases to minimise intersection pressure (a switch penalty discourages needless phase flapping).
4. Pre-train an LSTM queue forecaster and freeze it; concatenate its forecast onto the RL state to form the hybrid 56-dimensional observation.
5. Implement the three locked baselines (Webster, max-pressure, SUMO actuated) behind the same control interface for an apples-to-apples comparison.
6. Evaluate all controllers on seven KPIs across multiple scenarios and seeds, with a pre-registered statistical test plan, and run the mandatory ablations (with/without forecast, random-LSTM control).
7. Guarantee reproducibility through a provenance chain (data_version → lstm_version → run_id) and a weekly golden-hash regression test.
8. Provide a real-time visualisation layer (React dashboard + Unity 3D) streamed from the backend at 1 Hz, plus a replay browser.

4. User Stories

This is a simulation/research system; the "users" are the operator the policy emulates, the researcher running and evaluating experiments, and the people viewing the live/replayed visualisation.

US-1 — As a *traffic operator*, I want the controller to pick the signal phase that best relieves the current and predicted congestion, so that vehicles wait less.

US-2 — As a *traffic operator*, I want the controller to never command an unsafe or illegal phase change (respecting min/max-green, yellow, all-red and NEMA barriers), so that the intersection is always safe.

US-3 — As a *researcher*, I want to run any scenario × seed and get a byte-identical simulation, so that my experiments are reproducible.

US-4 — As a *researcher*, I want the DQN agent's training loss to decrease and its policy to improve over episodes, so that I can confirm it is actually learning.

US-5 — As a *researcher*, I want the LSTM forecast to be frozen and concatenated onto the state, so that I can ablate "with forecast" vs "without forecast" cleanly.

US-6 — As an *evaluator*, I want all seven KPIs computed identically (with the same warm-up cut-off) for every controller, so that the comparison is fair.

US-7 — As an *evaluator*, I want each controller compared on a shared per-episode seed, so that the paired statistical tests (Wilcoxon) are valid.

US-8 — As an *evaluator*, I want gridlocked/over-saturated episodes flagged ("gridlock-censored"), so that a controller cannot "win" by starving inflow.

US-9 — As a *researcher*, I want every result row to carry its full version chain (data/LSTM/run IDs + git SHA), so that any number can be traced back to its inputs.

US-10 — As a *viewer*, I want to watch the live intersection (vehicles + current phase + running KPIs) in a browser dashboard and a 3D scene at 1 Hz, so that the system's behaviour is visible and explainable.

US-11 — As a *viewer*, I want to pick a finished run and replay it, so that I can review and compare controllers after the fact.

Sample Test Code for User Stories

The following are **real, passing tests from the project's test suite** (161 tests green), each mapped to a user story. They demonstrate that the architecture is implemented and verified, not merely designed.

US-2 — controller never selects an unsafe/illegal phase (tests/test_dqn.py):

```
def test_act_never_picks_masked_action():
    agent = DQNAgent(seed=1)
    # global-best action 0 is ILLEGAL here; legal set is {1, 3}
    agent.online = lambda _x: torch.tensor([[5., 4., 3., 2., 1., 0., -1., -2.]])
    mask = np.array([False, True, False, True, False, False, False, False])
    assert agent.act(np.zeros(20), mask, epsilon=0.0) == 1      # greedy = best LEGAL
    for _ in range(50):
        assert mask[agent.act(np.zeros(20), mask, epsilon=1.0)] # even when exploring,
                                                                # only legal actions
```

US-4 — the DQN actually learns (loss decreases) (tests/test_dqn.py):

```
def test_loss_decreases_on_fixed_batch():
    agent = DQNAgent(obs_dim=20, seed=0)
    batch = Batch(obs=torch.randn(32, 20), action=torch.randint(0, 8, (32,)),
                  reward=torch.randn(32), next_obs=torch.randn(32, 20),
                  done=torch.zeros(32), next_mask=torch.ones(32, 8, dtype=torch.bool))
    first = agent.learn(batch)
    for _ in range(200):
        last = agent.learn(batch)
    assert last < first      # MSE TD-loss drops as the net fits the frozen targets
```

US-3 — runs are deterministic / reproducible (scripts/repro_smoke.py, golden hash):

```
# Re-run the fixed reference episode (SCN-02, seed 0, 300 s, Webster) and compare the
# SHA-256 of its normalized JSONL stream to the committed golden hash.
new_hash = sha256_of_normalized_jsonl(run_reference_episode())
assert new_hash == GOLDEN_HASHES["reference"] # byte-identical -> reproducible (exit 0/1)
```

US-6 — KPIs computed identically with a fixed warm-up window (tests/test_kpi_extractor.py):

```
def test_extract_kpis_applies_warmup_and_matches_hand_computation():
    kpis = extract_kpis(jsonl_path=synthetic_trace, tripinfo_path=synthetic_tripinfo)
    # KPIs are computed only over the [300, 3600] s window, discarded identically for
    # every controller; each metric is checked against a hand-computed value.
    assert kpis["avg_wait"] == pytest.approx(EXPECTED_WAIT, abs=1e-6)
    assert kpis["throughput"] == pytest.approx(EXPECTED_THROUGHPUT)
    assert kpis["gridlock_censored"] is False
```

5. Typical User Flows

Flow A — Train a controller

1. Select a scenario YAML and a seed → the route generator builds a deterministic .rou.xml.
2. The training loop runs episodes: at each 10 s step the env returns the 56-dim hybrid state + legal-action mask → the DQN picks a phase → SUMO applies it (yellow/all-red enforced) → reward and next state are returned and stored in the replay buffer.
3. Every 25 episodes validation runs; checkpoints (with version string) are written every 50.
4. On completion the trained checkpoint and a training report are produced.

Flow B — Evaluate & compare controllers

1. For each (scenario, seed) the evaluation runner executes every controller on a *shared* SUMO seed (paired design).
2. Each episode emits a 1 Hz JSONL trace + SUMO trip-info XML → the KPI extractor computes the 7 KPIs over the warm-up-trimmed window.
3. Rows (with full provenance chain) are written to SQLite.
4. The analysis notebook runs the pre-registered Wilcoxon tests + Holm–Bonferroni correction and produces the plots/tables.

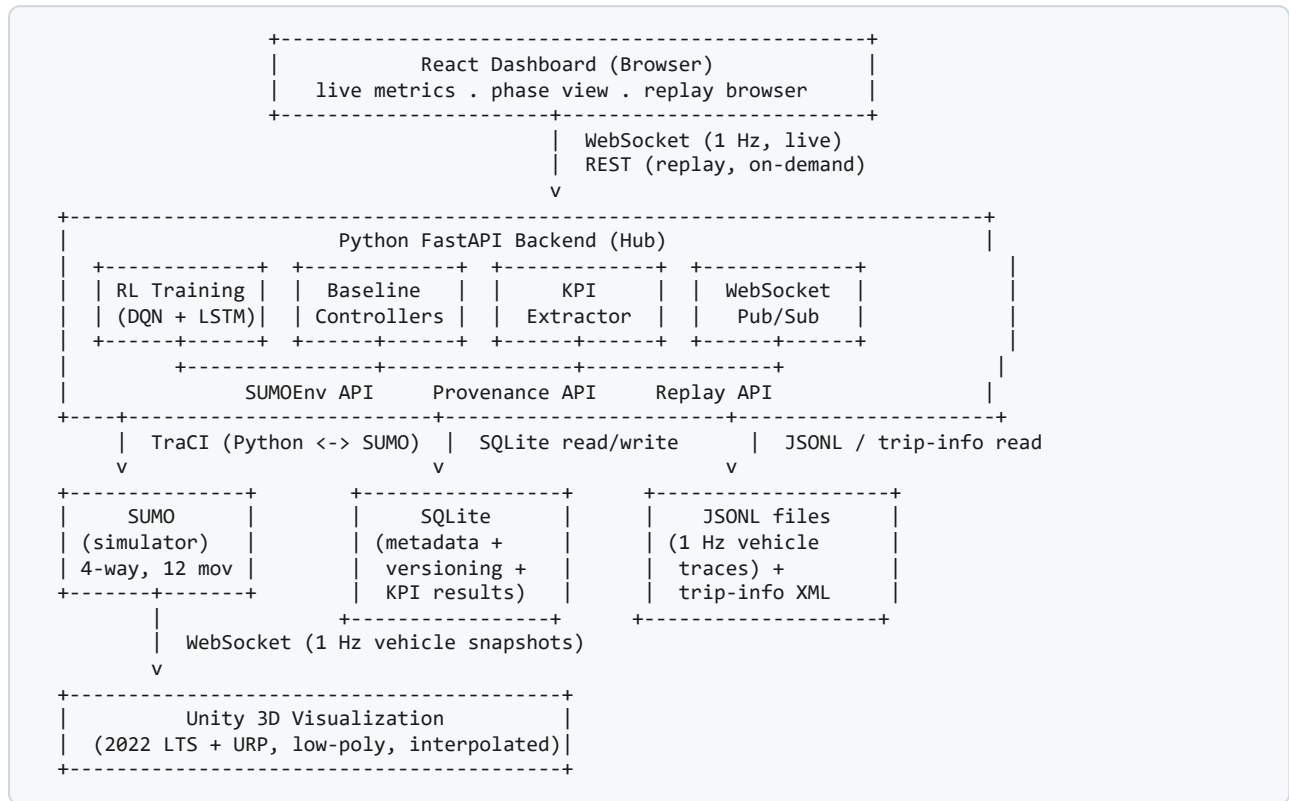
Flow C — Watch / replay (Phase 5)

1. Open the React dashboard → it subscribes to the backend WebSocket (1 Hz) → live metrics, current phase and vehicle snapshots render (client-side interpolation for smoothness).
2. To review a finished run: pick it from the replay browser → the backend streams the stored trace over REST → the dashboard and Unity scene play it back.

6. Project Entity Diagram

The system is five layers around a central Python (FastAPI) backend hub. The master architecture diagram (components and their connections) is shown first, followed by the persistence data model and the reproducibility provenance chain.

6.1 System architecture (components)



6.2 Persistence data model (SQLite, WAL mode)

Entity	Key fields	Relationships
scenario	scenario_id, demand profile, arrival rates, duration	1 → many runs
experiment_run	run_id (UUID), algorithm, seed, data_version, lstm_version, git_sha	belongs to scenario; 1 → many episodes
episode	episode_id, loaded/departed/arrived counts, insertion_backlog_fraction, gridlock_censored	belongs to run; 1 → 1 episode_kpi
episode_kpi	avg_wait, throughput, avg/max queue, p95 wait, per-movement fairness	belongs to episode
version chain	data_version, lstm_version, run_id	links every result to its inputs

6.3 Reproducibility provenance chain

```
SUMO network + scenario config    --> data_version (deterministic hash)
+ LSTM config + training seed    --> lstm_version  (in the checkpoint filename)
+ DQN config + training seed     --> run_id       (UUID; records all inputs)
```

7. Implementation

7.1 Technologies Used

Layer	Technology	Role
Simulation	SUMO 1.27 + TraCI / libsumo	Microscopic 4-way traffic simulation, driven from Python
Core language	Python 3.11	Environment, agents, baselines, metrics, backend
RL / ML	PyTorch 2.x (CPU)	DQN agent + frozen LSTM forecaster
Env API	Gymnasium	Gym-style reset/step/observe/close wrapper around SUMO
Backend hub	FastAPI + WebSockets	1 Hz live streaming, REST replay endpoints
Persistence	SQLite (WAL) + JSONL + SUMO trip-info XML	Run metadata, KPI results, vehicle traces, provenance
Dashboard (Phase 5)	React	Live metrics, phase view, replay browser
3D visualization (Phase 5)	Unity 2022 LTS + URP	Live/replayed low-poly 3D intersection
Statistics	SciPy / NumPy	Wilcoxon signed-rank + Holm–Bonferroni, bootstrap CIs
Tooling	pytest, black, ruff, Git/GitHub	161 unit/integration tests, formatting, lint, version control + CI

Deliberately excluded (single-user, simulation-only scope): no cloud services, no containerisation, no multi-user/auth, no real hardware integration. These are documented as future work.

Generated as the architecture report for the Smart Traffic Intersection Management System (replication-plus-adaptation of MPLight, Chen et al. 2020). Code & tests: github.com/Yuvalbrs/The-Smart-Traffic-Light-Project.